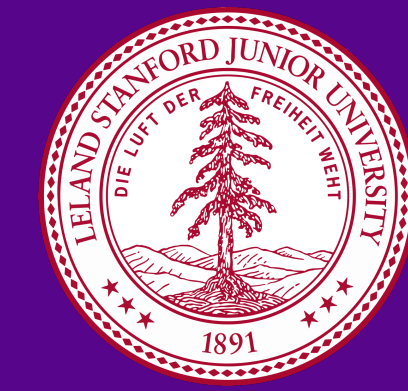


**NYU**Center for
Data Science

Iterative Tool Calling for Clinical Question Answering

Alexander Pegot-Ogier, Nikolas Prasinou, Sergey Sedov, Yash Jadhav
Mentor: Dr. Narges Razavian**NYU Langone
Health**

Context & Motivation

The Challenge: LLMs are increasingly adopted in complex healthcare applications. Structured EHR data is usually distributed across many tables, and simple LLM calls frequently fail to execute correct commands and hallucinate answers [4]. Agentic frameworks that enable iterative answer refinement are getting more popular. However, a proper evaluation of LLM Agents on complex Text-to-SQL clinical questions is missing, which results in vague understanding of limitations and bottlenecks of these approaches.

Our Goal: Implement Text-to-SQL tool and LLM Judges in OctoTools framework. Evaluate on the QA dataset that involves complex EHR data. Identify the main bottlenecks and suggest improvements. We aim to answer the following questions:

- What context is needed for an agent to run better?
- To which extent does iterative tool-calling help?
- Do agentic frameworks improve performance for smaller model sizes? What is the gap between smaller open-source and larger closed-source API models?
- What is the quality-compute trade-off? Does separation of reasoning and code writing into multiple prompts and LLM calls help?

OctoTools Framework

We utilized OctoTools [1], a training-free agentic framework that orchestrates external tools through a Planner–Executor workflow.

- Planner:** Decomposes the query into actions.
- Executor:** Runs SQL and returns results/errors.

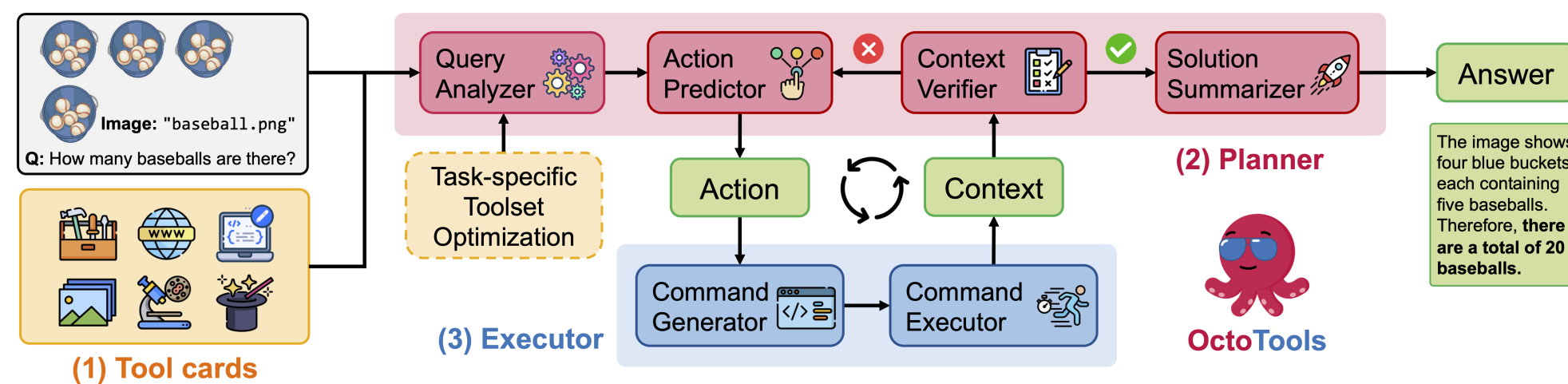


Figure 1. The Planner-Executor Feedback Loop

MIMIC-IV: EHR and QA Datasets

We built a PostgreSQL backend using the public MIMIC-IV demo subset [2] (100 patients), separating **hosp** (hospital stay) and **icu** (monitoring) modules.

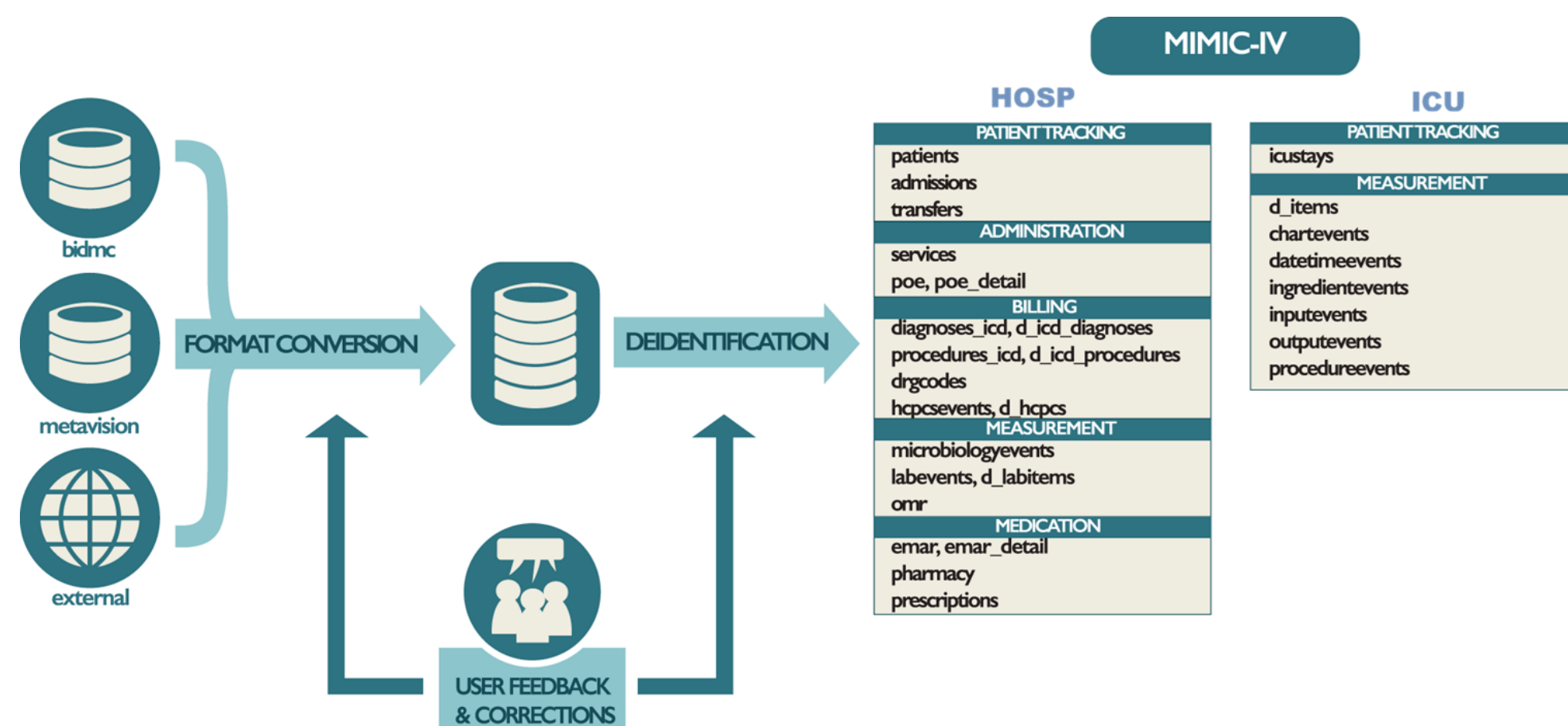
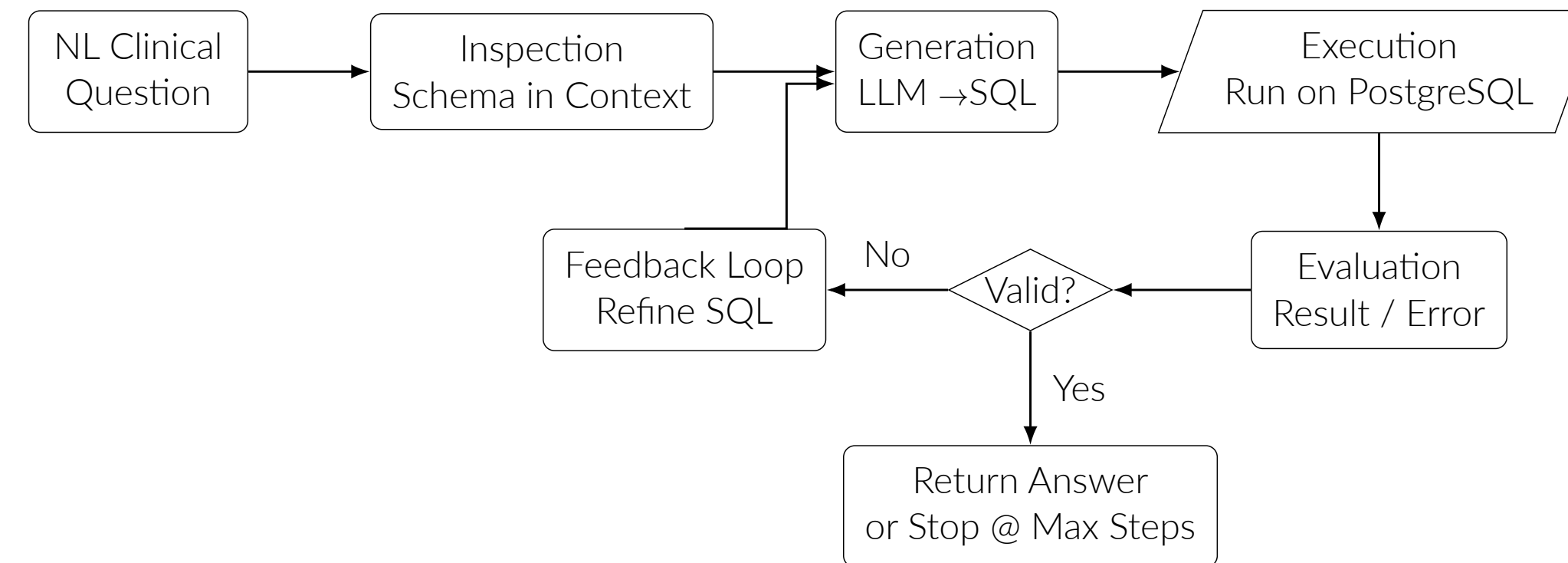


Figure 2. MIMIC-IV Database

NL to SQL Tool with Iterative Refinement

To bridge natural language and the database, we developed an OctoTools integration that inspects schema definitions, generates SQL, and executes queries against the backend. This process is governed by an iterative self-correction loop: the agent analyzes execution results or errors, reasons over failures, and automatically refines its queries until a valid answer is derived or the iteration limit is reached.



Evaluation Methodology

We evaluated our approach on 100 samples from the public MIMIC-IV-Extr-Instr QA dataset [3]. We focused on these variables:

- Number of iterations: how many times agent can refine its SQL code
- Different model sizes: Gemini 2.5 Flash-Lite & Flash, Gemma 12B
- Context provided to agent: textual table descriptions
- OctoTools vs. Custom SQL agent: does separation of planning and execution LLM calls help equally for different model sizes? what is the quality-compute trade-off

Results: Textual Table Descriptions

We analyze the effect of augmenting the PostgreSQL schema with textual table descriptions on clinical question answering performance. Results show that schema context interacts strongly with the reasoning framework: while textual descriptions improve performance for simpler SQL-based agents, they do not consistently benefit the OctoTools agentic framework. This suggests that static schema context and iterative agentic planning contribute differently to performance and may not be complementary.

Table 1. OctoTools performance without textual table descriptions across iteration budgets.

Model	Max steps	EM (%)	Judge (%)	Human (%)	API Calls	Time
Gemini 2.5 Flash	1	20	30	–	7.95	42.41s
Gemini 2.5 Flash	2	21	36	–	9.91	60.66s
Gemini 2.5 Flash	3	12	28	–	11.22	67.68s
Gemini 2.5 Flash	4	13	30	–	12.56	76.76s
Gemini 2.5 Flash-Lite	1	32	45	50	8.00	19.53s
Gemini 2.5 Flash-Lite	2	24	47	51	9.90	21.40s
Gemini 2.5 Flash-Lite	3	29	49	52	10.97	37.21s
Gemini 2.5 Flash-Lite	4	30	46	–	12.70	48.10s

Table 2. OctoTools performance with textual table descriptions.

Model	Max steps	EM (%)	Judge (%)	API Calls	Time
Gemini 2.5 Flash	2	13	31	9.96	65.46s

Results: OctoTools vs Custom SQL agent

We compared OctoTools against a custom SQL agent baseline (no planner/executor) to quantify the value of the OctoTools workflow.

Table 3. Custom SQL agent performance with textual table descriptions.

Model	Max steps	EM (%)	Judge (%)	API Calls	Time
Gemini 2.5 Flash	1	27	39	1.97	8.48s
Gemini 2.5 Flash	2	28	41	2.29	6.96s
Gemini 2.5 Flash	3	32	47	2.64	8.30s
Gemini 2.5 Flash	4	30	46	2.85	8.57s
Gemini 2.5 Flash-Lite	1	20	27	1.79	2.71s
Gemini 2.5 Flash-Lite	2	26	36	2.35	3.79s
Gemini 2.5 Flash-Lite	3	25	34	2.54	3.72s
Gemini 2.5 Flash-Lite	4	24	33	2.85	4.74s
Gemma 3 12B	1	17	20	1.34	6.41s
Gemma 3 12B	2	6	8	1.45	7.72s

Table 4. Custom SQL agent performance without textual table descriptions.

Model	Max steps	EM (%)	Judge (%)	API Calls	Time
Gemini 2.5 Flash	1	27	33	1.92	8.47s
Gemini 2.5 Flash	2	31	45	2.39	8.49s
Gemini 2.5 Flash	3	30	41	2.59	8.06s
Gemini 2.5 Flash	4	30	44	2.83	8.49s
Gemini 2.5 Flash-Lite	1	20	27	1.77	3.03s
Gemini 2.5 Flash-Lite	2	22	33	2.20	3.28s
Gemini 2.5 Flash-Lite	3	25	30	2.45	3.70s
Gemini 2.5 Flash-Lite	4	25	25	2.90	3.77s
Gemma 3 12B	1	14	16	1.32	4.64s
Gemma 3 12B	2	6	9	1.47	5.16s

Key Findings

- Model-Framework Interaction:** Iterative agentic tool calling benefits smaller models the most. Gemini 2.5 Flash-Lite achieved its best performance under OctoTools (up to 49%), while the larger Gemini 2.5 Flash performed more consistently with the optimized SQL agent baseline.
- Context Trade-offs:** Textual schema descriptions had opposing effects. They improved the Custom SQL Agent (helping Flash reach 47%) but degraded OctoTools performance (dropping Flash to 31%), suggesting that excessive text may distract the agentic planner.
- Benefit of Iteration:** Across both setups, iterative self-correction consistently improved accuracy over single-shot attempts, with performance typically maximizing at 2–3 steps.

References

- [1] Lu, Pan et al. *OctoTools: An agentic framework with extensible tools for complex reasoning*. arXiv, 2025.
- [2] Johnson, Alistair et al. *MIMIC-IV clinical database demo*. 2023.
- [3] Wu, Z. et al. *MIMIC-IV-Ext-Instr: A dataset of 450k+ ehr-grounded instruction-following examples*. 2025.
- [4] Ji, Ziwei et al. *Survey of hallucination in natural language generation*. ACM Computing Surveys, 2023.